LeetCode Practice

1 Easy: Trie

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

1
EASY

Patterns (in order)

1. Trie (1)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

1 questions | tap a title to jump to it

Easy questions (1) Go >

Trie (1) Go >

E 1. Longest Common Prefix LC 14

Easy questions (1)

EASY | TOPIC 1 OF 1

Trie

1 Easy question start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Longest Common Prefix LC 14

EASY | Trie | LeetCode #14 | 1 of 1 | 1/1

Longest Common Prefix

QUESTION

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1	
Input	`strs = ["flower","flow","flight"]`
Output	`"fl"`

Example 2	
Input	`strs = ["dog","racecar","car"]`
Output	`""`
Explanation	*There is no common prefix among the input strings.*

CONSTRAINTS

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] consists of only lowercase English letters if it is non-empty.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the longest prefix shared by ALL strings in the array.

If none exists, return an empty string. Right?"

"A few quick questions:

Can the array be empty? No — length >= 1. Can a string be empty? Yes.

Characters are lowercase English letters only."

"Let me walk: ['flower','flow','flight']: fok lok o vs o vs i -> mismatch -> 'fl' ok

['dog','racecar','car']: d vs r mismatch immediately -> '' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for L in range(len(strs[0]),_0_-_1,_-1):
    if every string starts with strs[0][:L]:
        return that prefix
```

TIME	SPACE
O(n * L^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Re-compares the same leading characters for every candidate length L (n strings, L the shortest length). Scanning column by column and stopping at the first mismatch is O(n * L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n log n) sort — we can skip it entirely.

Vertical scanning: take the first string as reference. For each character position,

check that every other string has the same character at that position.

The first mismatch (or any string running short) terminates the prefix.

Time: O(n*m) where n=number of strings, m=min string length. Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _14_LongestCommonPrefix:
    def longestCommonPrefix(self, strs):
        if not strs:
            return ""
        for i, ch in enumerate(strs[0]):
            for s in strs[1:]:
                if i >= len(s) or s[i] != ch:
                    return strs[0][:i]
        return strs[0]
```

PHASE 5 TEST & DEBUG

"Let me trace longestCommonPrefix on ['flower','flow','flight']."

Vertical scan: fix first word 'flower' as reference

i=0: 'f' — check all: flow[0]='f' ok, flight[0]='f' ok -> continue

i=1: 'l' — flow[1]='l' ok, flight[1]='l' ok -> continue

i=2: 'o' — flow[2]='o' ok, flight[2]='i' -> stop, LCP='fl'

Return "fl" ok

Edge: ["dog","racecar"] -> first char 'd'!='r' -> return ""

WHY vertical scan: stops at first mismatch without checking full strings.

Complexity: O(S) where S = total chars in shortest string × count.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*m)	O(1)
n strings × m min length	no extra allocation

Sort-then-compare is O(n log n) — vertical scan is strictly faster.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: if the array is huge but strings are short, binary search on prefix length:

check if strs[0][:mid] is a prefix of all strings — O(n log m) total.

Follow-up: Trie — insert all strings and walk until a branch point.

IN YOUR APP | Expose | Editorial | Community